That breaks the test double you use in the controller test to bypass the action—which I admit is more fumbling in the codebase than I thought was necessary when I started this example:

security/05/spec/controllers/projects_controller_spec.rb

```
require "rails_helper"

RSpec.describe ProjectsController, type: :controller do
  let(:user) { create(:user) }

  before(:example) do
    sign_in(user)
  end

  describe "create" do
    it "calls the workflow with parameters" do
      workflow = instance_spy(CreatesProject, success?: true)
      allow(CreatesProject).to receive(:new).and_return(workflow)
      post :create,
        params: {project: {name: "Runway", tasks: "start something:2"}}
      expect(CreatesProject).to have_received(:new)
        .with(name: "Runway", task_string: "start something:2", users: [user])
    end

  end
end
```

A similar change to the test double needs to be made to the add_project_spec workflow, which also uses a test double for one test.

After all that, you have basic user and role authentication in the system. Now you need to protect against a couple of attacks that require the user to not use the application's UI directly.

> **Prescription 33** Adding user authentication can be very disruptive to existing tests. Try to get the basic infrastructure in place early.

Protection Against Form Modification

There is at least one blind spot in the user and role protection. The project show page has a form that submits a new task. That form is submitted to the TasksController, which doesn't handle any user-access control. The use case here is a malicious user not going through the web UI but rather creating his own HTTP request and pointing it at the server.

There are two important issues here, at least from my perspective as Rails Testing Author Guy. First is the habit of noticing when you're using a resource that's being accessed as a result of a user request as opposed to being stored